Orca ALab Edition

Architecture, Upstream Comparison, and TrustJS Roadmap

As-built execution model and strategic migration options

Status	Current-state architecture report and decision roadmap
Date	July 21, 2026
ALab snapshot	`d3ba1bcb17324714a838b2970e04fcd96989e84f` `1.4.147-fork.1`
Upstream snapshot	`ac909c8d837621b5b64c70a18af05fed7eb10fad` `1.4.150-rc.0`
Trust snapshot	`10606c9efa9be25cb798c350a55619234a7ce064`
Evidence basis	Pinned source and build-entry analysis; no fresh cross-platform application run is claimed.

Preferred direction. Keep TypeScript/JavaScript as the business-logic authoring surface, adopt TrustJS through measured admission gates, and retain manual Rust porting for terminal, PTY, crypto, protocol, privileged-boundary, and proven performance work.

Contents

Executive conclusion

Orca ALab is an Electron application with a React/TypeScript product shell and several deliberately placed Rust execution islands. It is not currently a native Rust application. TypeScript still starts Electron, creates windows, owns the preload and IPC surfaces, coordinates repositories and sessions, deploys the SSH relay, integrates providers, and renders the product UI. Rust owns the local terminal daemon, the aterm terminal engines, a broad but bounded set of pure/business operations, and selected Git, crypto, persistence, and protocol paths.

The central difference from upstream is the terminal and portable-core data plane:

- **ALab:** native Rust daemon + native Node-API add-on + aterm WASM + shared Rust dispatch exposed through Node-API and WASM.
- **Upstream:** forked Node daemon + `node-pty` + xterm.js/headless xterm + TypeScript business logic.

Both editions still literally boot an Electron main-process JavaScript bundle, bridge through a sandboxed preload, and mount a React renderer. ALab has replaced important engines below that shell; it has not replaced the shell.

For the forward roadmap, **option 2 should be the default for business logic: keep authoring features at the TypeScript/JavaScript level and adopt TrustJS in stages.** Today that means TrustJS-assisted differential validation while Node remains the production engine. It does not yet mean executing Orca under TrustJS. Manual Rust porting should continue only for the places where Rust is intrinsically valuable: PTYs, terminal parsing/rendering, crypto, protocol and byte processing, privileged host boundaries, stable state machines, and measured performance bottlenecks.

That yields the intended hybrid:

```
React/Electron UI and feature composition                TypeScript
Pure, provider-neutral business logic                    TS/JS → TrustJS track
Host effects, provider SDKs, filesystem, SSH adaptation  TypeScript ports/adapters
Terminal, PTY, crypto, hot parsers, native persistence   Rust
```

TrustJS is promising but not a production replacement today. Its current product-level description is an in-development Rust JavaScript interpreter and differential apparatus. It does not parse TypeScript, load application modules, provide Node APIs, expose a stable Orca-callable ABI, ship Node-API/WASM bindings, lower to Trust-IR, or emit validated native code. Those are roadmap gates, not present-tense capabilities.

Scope and method

This report distinguishes three things that older migration documents sometimes blend together:

1. **As-built:** code reached by the current product's build and runtime entry points.
2. **Implemented but not product-wired:** Rust crates, native prototypes, or migration artifacts that compile or have tests but are not on the normal application path.
3. **Target:** the fully native and Trust-verified architecture described in migration plans.

The upstream comparison uses the locally fetched `upstream/main` ref. Its reflog records a fetch at 2026-07-21 19:00 PDT. No branch was checked out and no Internet-only state is assumed. The report is a source/build trace; it does not claim that a fresh application build or cross-platform runtime matrix was executed for this review.

1. The as-built ALab architecture

1.1 Process and trust-boundary map

```
OS launches Electron
│
├─ Electron main process — compiled TypeScript/JavaScript
│  ├─ application lifecycle, windows, IPC, services, providers
```

```
│  ├─ loads orca_node.node — Rust Node-API dynamic library
│  ├─ spawns/connects to orca-daemon — standalone Rust process
│  ├─ serves runtime RPC to CLI/mobile/web/headless clients
│  └─ deploys/manages the TypeScript Node relay on SSH hosts
│
├─ sandboxed preload — compiled TypeScript/JavaScript
│  └─ contextBridge exposes the typed window.api IPC surface
│
├─ Chromium renderer — React + compiled TypeScript/JavaScript
│  ├─ product UI, Monaco, pane lifecycle, feature state
│  ├─ Rust Git/dispatch and crypto WASM
│  └─ shared render worker
│     └─ aterm CPU/GPU WASM → OffscreenCanvas
│
└─ local terminal daemon — Rust
   ├─ authenticated local socket / Windows named pipe
   ├─ native PTYs and process lifecycle
   ├─ session-side aterm headless state and checkpoints
   └─ binary/NDJSON event and control protocol
```

The executable entry is still `out/main/index.js`, declared in `package.json` and built from `src/main/index.ts`. The main module binds the Rust dispatch surface before importing Electron. TypeScript then controls `app.whenReady()`, constructs `BrowserWindow`, and loads either the Vite dev URL or the packaged renderer HTML. The sandboxed preload publishes `electron` and `api` with `contextBridge`; the renderer calls React's `createRoot` and renders `App`.

1.2 What TypeScript owns

TypeScript is the control plane and product layer:

- Electron startup, single-instance behavior, windowing, menus, certificates, updates, service initialization, and shutdown;
- preload types and the large IPC API exposed to the sandboxed renderer;
- repository, worktree, session, agent, provider, and feature orchestration;
- PTY routing, flow control, batching, environment construction, shell policy, recovery, and daemon lifecycle;
- SSH connection management, relay deployment, remote filesystem/Git/process effects, and compatibility policy;
- runtime RPC used by the CLI, mobile/web clients, and headless `serve` mode;
- React, pane lifecycle, Monaco, editor/browser/source-control surfaces, and user interaction.

The `.ts` and `.tsx` files are not interpreted directly. electron-vite/Vite and TypeScript compile or bundle them to JavaScript for Electron main, preload, Chromium, CLI, web, worker, and relay contexts. This matters: ALab is a multi-context JavaScript product even where several algorithms behind those contexts are Rust.

1.3 How Rust is delivered and invoked

Rust form	Literal artifact and loader	Current role
Standalone native process	`orca-daemon` is built from `rust/crates/orca-daemon` and spawned by TypeScript	Local PTYs, process/session lifetime, session-side terminal state, checkpoints, socket protocol
Node-API dynamic library	`orca_node.node` is built from `native/orca-node` and loaded into Electron main/CLI	Native aterm headless API, Git parsers and operations, orchestration store, and aggregate pure/business dispatch
Renderer/worker WASM	aterm CPU and GPU WASM are imported by renderer code and normally hosted in a shared Web Worker	Per-pane parsing, terminal state, search/selection, and pixels on `OffscreenCanvas`
Portable business WASM	`orca-git-wasm` and crypto WASM are initialized in sandboxed or relay-safe contexts	Shared dispatch/Git rules in renderer and SSH relay; browser-safe crypto

Rust form	Literal artifact and loader	Current role
Direct Rust linking	The daemon links `orca-pty`, `orca-terminal`, and `orca-net` directly, plus `orca-winpipe` on Windows	Avoids Node-API on the daemon's hot path
C ABI/native shell prototype	`orca-ffi` and `native/orca-macos`	Prototype/target scaffold; not the normal electron-builder product path

The live Rust workspace has 26 members in `rust/Cargo.toml`, but workspace membership is not proof of product execution. The shipped roots are narrower: the daemon, the aggregate Node-API add-on, aterm WASM, Git/dispatch WASM, and crypto WASM.

The important business-logic seam is `orca-dispatch-seam.ts`. Main and CLI bind it to Rust through Node-API; renderer and relay contexts bind the same conceptual registry through WASM. The Rust registry currently spans roughly 79 pure-domain modules. This lets TypeScript retain host integration while one Rust implementation supplies portable rules to several JavaScript runtimes.

The boundary is intentionally coarse. A JavaScript caller sends a module and a serialized input to Rust and receives a serialized result. This is a much better FFI shape than crossing Node-API/WASM for every field or byte, but it also means the serialization contract is part of the architecture.

1.4 Literal development execution

`pnpm dev` executes this chain:

```
pnpm dev
  1. build native/orca-node → native/orca-node/orca_node.node
  2. build rust/crates/orca-daemon → rust/target/release/orca-daemon
  3. rebuild/check Electron-ABI native dependencies
  4. prepare isolated dev identity, data directory, CLI wrapper and debug port
  5. spawn electron-vite dev
  6. electron-vite builds/watches main, preload, workers and React renderer
  7. Electron loads out/main/index.js
```

The chain is visible in `package.json` and `run-electron-vite-dev.mjs`. The aterm, Git-dispatch, and crypto WASM bundles are not rebuilt by every `pnpm dev`; their explicit regeneration commands are `build:aterm-wasm`, `build:relay-wasm`, and `build:crypto-wasm`.

`pnpm start` is also not a clean build path. It ensures the Electron native runtime and starts `electron-vite preview`, assuming the JavaScript bundles and Rust/native artifacts already exist.

electron-vite has separate entries for main-process workers/sidecars, preload, the main renderer, and the coordinator renderer. The build therefore preserves Electron's process boundaries rather than producing one monolithic JavaScript program.

1.5 Literal packaged execution

The platform packaging scripts build the TypeScript bundles, native add-on, Rust daemon, CLI, relay, and web bundle before invoking electron-builder. The macOS commands additionally build their applicable native helpers. electron-builder places JavaScript in `app.asar`, but copies the native add-on and daemon as external resources because one must be loaded by the dynamic loader and the other must be executed.

At launch:

```
Orca ALab app executable
  → Electron runtime
  → app.asar/out/main/index.js
  → load Resources/orca_node.node
  → locate/spawn Resources/orca-daemon[.exe]
  → connect with a random token over a local socket/named pipe
  → create sandboxed BrowserWindow + preload
  → load out/renderer/index.html
```

```
  → initialize dispatch/crypto WASM and mount React
```

The product identity is deliberately separate from upstream: `com.stablyai.orca.staging` / `Orca ALab Edition` in `electron-builder.config.cjs`.

There is one build-consistency gap worth fixing: `build`, `build:release`, and `build:unpack` do not explicitly rebuild `orca-daemon`, while the platform packaging commands do. electron-builder rejects a missing required resource, but a generic build can consume an existing stale daemon. A single canonical artifact graph should make the daemon dependency explicit for every packaging entry point.

1.6 Literal local terminal execution

The normal local control path is:

```
React pane
  → preload IPC: pty:spawn / pty:write / pty:resize
  → TypeScript PTY router in Electron main
  → DaemonPtyAdapter / DaemonClient
  → authenticated daemon control socket
  → Rust orca-daemon
  → Rust orca-pty
  → native OS PTY
  → shell / agent process
```

Output returns on the event path:

```
native PTY bytes
  → Rust daemon read loop
  → Rust aterm headless state + checkpoint/session bookkeeping
  → daemon event socket (binary frames preferred; NDJSON control)
  → TypeScript daemon client + flow control
  → Electron IPC pty:data
  → preload
  → pane transport/controller
  → shared render worker
  → aterm WASM parses bytes and paints OffscreenCanvas
```

Input and resize travel in the opposite direction. The daemon uses separate control and event-stream connections with authenticated hello/version negotiation. The reviewed code's protocol is 1020 with minimum compatible version 1018; binary streaming was introduced at 1020.

The renderer first attempts the shared worker. If that path cannot initialize, it falls back to in-process aterm GPU and then CPU WASM. There is no xterm.js rendering fallback in ALab.

The daemon is detached so a normal UI exit can leave local sessions alive for warm reattachment. If daemon startup fails, there is no alternate Node daemon; the application degrades to the in-process TypeScript `LocalPtyProvider` backed by `node-pty`. Fresh terminals still work, but daemon persistence across app exit is lost.

Depending on the active features, one session can be represented by as many as three independent aterm instances:

1. native aterm in the daemon for session/checkpoint state;
2. native aterm through Node-API in Electron main for headless query/snapshot paths;
3. aterm WASM in the renderer for interaction and pixels.

They share an implementation lineage, not object identity. Feeding equivalent byte streams and maintaining snapshot/parity tests is therefore a real correctness, memory, and CPU concern.

1.7 SSH is a different runtime

The local Rust daemon does not own SSH PTYs. For remote work, Electron main uploads a content-hashed CommonJS relay bundle and launches it under Node on the remote host:

```
Electron main
  → ssh2 / SSH channel
```

```
  → uploaded relay.js under remote Node
  → remote node-pty, filesystem, Git and agent processes
  → framed channel back to Electron main
```

The relay embeds Rust Git/dispatch WASM for portable parsing and decisions, but the remote I/O and process host remains TypeScript/Node. That is an intentional portability property: ALab does not need to build and upload a native add-on for every SSH host architecture. Any future business-logic execution policy must preserve this use case; a desktop-only Node-API solution is incomplete.

1.8 CLI and headless service

The CLI is also TypeScript compiled to `out/cli/index.js`. Packaged wrappers run the Electron binary with `ELECTRON_RUN_AS_NODE=1`; the CLI binds the same Rust dispatch surface where available and talks to the app's runtime RPC service.

`orca serve` enters the same Electron main bundle without a normal main window. It still initializes services, the local PTY provider/Rust daemon, headless terminal support, and runtime RPC. Some browser work can still require offscreen Chromium windows. "Headless" therefore means no normal UI, not "no Electron."

2. How upstream Orca runs

At the reviewed upstream commit, Orca has the same high-level Electron shape:

```
OS → Electron main JavaScript
   → sandboxed preload
   → React renderer
```

Its terminal implementation below that shell is different:

```
React/xterm.js pane
  → preload IPC
  → Electron main daemon adapter
  → forked daemon-entry.js with ELECTRON_RUN_AS_NODE=1
  → TypeScript DaemonServer
  → node-pty
  → shell

shell output
  → node-pty
  → @xterm/headless state + serialization in Node daemon
  → main/preload IPC
  → renderer xterm.js + WebGL/DOM/canvas addons
```

Upstream's `pnpm dev` ensures the Electron native runtime and starts electron-vite; it does not compile ALab's Rust add-on or daemon. Its electron-vite configuration emits `daemon-entry.ts` as a separate JavaScript bundle. Main uses Node's `fork()` with the Electron executable running in Node mode, and the child uses `node-pty`. The renderer constructs xterm.js `Terminal`, Fit, Search, Serialize, Unicode, WebLinks, and WebGL-related addons. The daemon separately uses `@xterm/headless` to maintain persistent terminal state and snapshots.

Upstream has native npm dependencies, including `node-pty`, but no tracked `rust/` product workspace at this ref. Its ordinary feature and business logic remains TypeScript/JavaScript.

3. ALab versus upstream

Dimension	ALab edition	Upstream Orca
Product shell	Electron main + sandboxed preload + React renderer, mostly TypeScript	Same fundamental shell
Local terminal daemon	Standalone Rust `orca-daemon`	Forked Node/Electron JavaScript daemon

Dimension	ALab edition	Upstream Orca
Local PTY	Rust `orca-pty`; `node-pty` only for degraded in-process fallback	`node-pty` in the Node daemon
Session-side terminal	aterm linked into Rust daemon	`@xterm/headless` in Node daemon
Renderer terminal	aterm CPU/GPU WASM, normally in shared worker + OffscreenCanvas	xterm.js and addons, including WebGL path
Main/CLI native logic	Aggregate Rust Node-API add-on	TypeScript/JavaScript and native npm dependencies
Renderer/relay portable core	Rust WASM dispatch/Git/crypto	TypeScript/JavaScript
SSH host	TypeScript Node relay + `node-pty`, with Rust WASM for selected pure logic	TypeScript Node relay + `node-pty`
Failure mode	Rust-daemon failure degrades to in-process TS provider; no Node-daemon twin	Node daemon is the normal daemon
Build complexity	Node/pnpm + Rust + WASM + native add-on + per-platform resource packaging	Node/pnpm + Electron/native npm rebuilds
Default product compiler for Rust	stable Rust 1.96 product path; optional Trust toolchain in selected scripts	No first-party Rust product build at this ref
Native desktop shell	SwiftUI/C-ABI prototype exists but is not shipped	Electron product
Verification claim	Terminal and migration-specific evidence; not whole-app or whole-build Trust verification	Conventional TypeScript/Rust-free product test surface at this ref

3.1 Repository relationship

The repositories are still closely related, not independent products:

- merge base: `d9d939a33b5858495ffb33489a952f1ac9293610`;
- ALab-only commits from that base: 883;
- upstream-only commits from that base: 99;
- latest explicit upstream merge in ALab: `b206b314790244e85565a5f1bb54afb11f6e7cdc`;
- 9,095 paths exist in both snapshots and 7,722 of those have identical blobs, approximately 85% of shared paths.

ALab has 9,662 fork-only paths and upstream has 471 upstream-only paths, but 8,190 of the ALab-only paths are vendored Rust dependencies. Raw line or path counts therefore wildly overstate how much of the product was rewritten. The more useful conclusion is architectural: most of the Electron/product shell is still shared, while ALab has replaced or wrapped selected lower layers.

This shape has two strategic consequences:

1. continued upstream merging remains valuable because most product work is still shared;
2. every manual semantic port increases the fork's merge and parity burden, so a module should not move to Rust merely because it can.

3.2 Current caveats and documentation drift

- `docs/rust-migration/architecture.md` is a target architecture, not the current runtime. Sections that describe the daemon as TypeScript or the native shell as the product are historical or aspirational.
- `rust/README.md` says 18 workspace crates and records older aterm provenance; the live manifest has 26 members and the aterm gitlink is `97b9dcbe5f6cf8619f3228d4367a7dca0ac2ff20`.
- The normal product daemon build explicitly selects stable Rust 1.96 and clears Trust-only flags. Selected add-on/WASM scripts accept `ORCA_RUST_TOOLCHAIN=trust`, but that is not the default shipping compiler. Consuming aterm code that has its own proof campaign does not make the whole Orca binary Trust-verified.
- Rust-daemon Windows named-pipe code compiles, but its source says real Windows end-to-end runtime exercise

remains incomplete.
- The daemon release profile uses `panic=abort`: an uncaught Rust panic ends the daemon. Recovery restarts the process, not an individual session.
- The Node-API add-on is no longer merely an optional terminal acceleration. It carries orchestration persistence and broad dispatch logic, making artifact availability effectively a product requirement at startup.

4. What TrustJS actually is today

Trust is a verification-oriented fork of Rust. TrustJS is a set of Rust crates inside that repository. Its current execution is:

```
pinned Test262 JavaScript case
├─ Node subprocess + embedded trace driver ─┐
├─ Bun subprocess + embedded trace driver ──┼─ compare ObservableTrace
├─ trust-js-sem in-process Rust semantics ──┤
└─ trust-js-interp in-process Rust engine ──┘
```

The current faithful engine parses ECMAScript Script source into a Rust AST, walks that AST in a Rust interpreter, and returns either an `ObservableTrace` or an explicit `NoCoverage` refusal. Its public API is shaped for corpus cases—`evaluate_case(includes, body, strict)`—rather than application modules. Each evaluation currently creates and joins a fresh thread with a 32 MiB stack. The value heap is an arena bounded by per-evaluation resource caps; there is no long-lived runtime garbage collector.

The TrustJS crates currently do **not** depend on Trust-IR, Clean, `trustc`, or the autoformalization stack. The faithful interpreter explicitly resolves and proves nothing. It is valuable because it refuses unsupported behavior and is differentially measured, not because it already produces proofs.

4.1 Published evidence

The latest published S1c dashboard at the reviewed snapshot reports:

Evidence	Result
Frozen Test262 S0 cases	35,346
Node/Bun runs	68,549
Node/Bun trace-equal	67,510 (98.48%)
Node/Bun divergent runs	1,037, all classified
TrustJS faithful-tier runs covered and equal	38,406
TrustJS faithful-tier divergent	0
TrustJS faithful-tier `NoCoverage`	30,141
Independent-semantics runs covered and equal	22,122
Independent-semantics divergent	0

The parser's published gate reports 99.6468% coverage on its admitted verdict lane, with disagreements audited and unsupported inputs explicit. The newest TrustJS source commit adds S1d regular-expression work, but its commit message says gate verification is in progress and there is no corresponding published S1d evidence/ratchet row. S1c is therefore the latest authoritative execution claim.

These are good engineering results and a useful zero-wrong-trace discipline. They are not evidence that TrustJS runs 56% of Orca. S0 intentionally excludes modules, async, shared memory, `Intl`, `Temporal`, cross-realm behavior, host GC, and other application/runtime surfaces. Test262 coverage cannot be projected onto an Electron application.

4.2 Missing product capabilities

Needed by Orca	TrustJS at this snapshot
TypeScript/TSX parsing	Not implemented; current parser accepts JavaScript

Needed by Orca	TrustJS at this snapshot
Named application modules and imports	M2 scope, not current code
Promise/event loop/async/await	M2 scope, not current code
`node:` APIs and npm packages	M2 pure-JS subset planned; native-addon compatibility explicitly deferred
Persistent callable runtime	No product API; current entry is case evaluation
Host callbacks/capability ABI	Not implemented for Orca
N-API, WASM, or standalone runtime artifact	No product-consumable runtime is shipped; the only executable is the corpus-focused differential harness
Long-lived GC/object model	Planned after the current arena/test model
Trust-IR lowering	M3/M4 boundary, not current code
Autoformalized native output and install certificates	M4 target, not current code
Cross-platform product CI and packaging	Current harness defaults are Linux-user-specific; no Orca packaging lane
Performance evidence	No TrustJS application benchmarks are published

The Trust README consequently still classifies TrustJS as design/BUILD work and not a shipped frontend. That is the correct product-level description despite the substantial M0/M1 implementation.

4.3 TypeScript is not proof input

An Orca adoption would need a pinned TypeScript-to-JavaScript transform before the current TrustJS parser could consume code. The artifact identity must bind at least:

- TypeScript source hash;
- compiler/transpiler version and exact options;
- emitted JavaScript hash;
- TrustJS revision and runtime build identity;
- admitted capability/effect manifest;
- differential corpus and evidence revision.

Type annotations, JSDoc, names, and comments may propose intent; they cannot assert proof facts or narrow the comparison corpus. This is Trust's zero-authority-frontend rule. Differential equality is bounded behavioral evidence. Only a future Trust-IR/Clean lane can carry the corresponding proof authority.

4.4 Do not conflate TrustJS with earlier port tooling

`trust-ts-embed`, `trust-formalize`, and `tools/ts2rust` are useful precursors, but they are not the current TrustJS runtime. In particular, ts2rust creates or assesses a separate Rust candidate, compares it against Node on a finite corpus, and asks Trust about Rust safety. It is a migration assistant, not general TypeScript semantics and not an integrated Orca execution engine.

5. Roadmap options

5.1 Decision matrix

Criterion	Option 1: continue manual Rust porting	Option 2: TrustJS-first hybrid
Can ship new product logic today	Yes, after port/parity work	Yes in TypeScript; TrustJS shadowing only today
Feature iteration speed	Lower for every manually translated module	Highest: one maintained TS/JS source
Native performance certainty today	High for well-designed ports	Unknown until later native tier; interpreter unbenchmarked
Existing Orca integration seam	Mature Node-API, daemon, WASM, dispatch	Must build callable/runtime and packaging seams

Criterion	Option 1: continue manual Rust porting	Option 2: TrustJS-first hybrid
Browser/renderer/SSH portability	Established through WASM and TS relay	Requires a WASM or portable artifact before primary remote use
Behavioral-fidelity cost	Per-module translated tests and dual-run corpus	Central engine corpus plus per-export admission corpus
Proof status today	Rust is not automatically Trust-verified; proof lane remains separate	Differential evidence only; native/proof authority is M4+
Upstream merge burden	Grows with every deleted/replaced TS module	Lower while feature source remains TS/JS
Main risk	Feature capacity consumed by translation and semantic drift	Betting product cutover on an immature runtime if gates are skipped
Best fit	Data plane, privileged boundaries, measured hot paths	Pure/provider-neutral business logic and fast feature work

5.2 Option 1 – continue manual TypeScript-to-Rust porting

This option extends the pattern ALab already proved: define a coarse boundary, port the behavior and tests, expose it through Node-API/WASM/daemon protocols, dual-run against the TypeScript implementation, cut over, then remove the hand-maintained twin after an explicit stability window.

Phase R1 – make current ownership truthful

- Generate an as-built inventory from actual import/build roots, not crate existence or old migration checklists.
- Classify every candidate as UI, host/effect adapter, pure business rule, performance data plane, or unused/test-only Rust port.
- Fix the generic-build daemon dependency and establish macOS, Linux, Windows, WSL, and SSH artifact matrices.

Gate: every shipped Rust root is reproducibly built, packaged, loaded, and smoke-tested on its applicable hosts; documentation names current ownership.

Phase R2 – harden the Rust substrate already in production

- Exercise Windows named-pipe daemon startup, reconnect, upgrade, crash, and uninstall paths on real Windows.
- Test daemon protocol downgrade/upgrade and token/socket isolation.
- Measure the cost of multiple aterm state instances and eliminate unnecessary duplicate processing where semantics permit.
- Preserve the Node relay/WASM shape for SSH rather than assuming a desktop add-on exists remotely.

Gate: crash/reconnect/session-persistence and cross-host parity pass with bounded resources and actionable telemetry.

Phase R3 – port only justified vertical slices

Require at least one concrete reason per module:

- measured CPU, latency, allocation, or startup improvement;
- privileged memory/process boundary;
- protocol/parser totality or security value;
- stable cross-context implementation needed in native and WASM forms;
- mature state machine whose churn is low enough to recover the port cost.

Freeze a versioned request/result schema first. Capture TypeScript behavior with unit, property, fuzz, recorded-production, and mutation corpora. Run both implementations, including provider, GitLab/GitHub, SSH, WSL, and Git 2.25 compatibility cases where relevant.

Gate: zero unexplained divergence, acceptable performance, all applicable host contexts green, and an atomic rollback path before primary cutover.

Phase R4 – decide separately whether to pursue a native shell

The existing SwiftUI/C-ABI code is not a consequence of porting one more business module. A native shell requires its own product decision, especially for Monaco/editor capability, accessibility, i18n, Linux, and Windows. Do not let a backend-port program silently become a full UI rewrite.

Option 1 assessment

This is deployable and appropriate for the existing Rust data plane. As the default for ordinary business features, however, it spends engineering effort twice—first to implement the feature, then to translate and prove parity—and it increases the cost of every upstream merge. It should be a selective tool, not the organizing principle for the entire application.

5.3 Option 2 – TrustJS-first hybrid (preferred)

The practical policy is **TrustJS-assisted now, TrustJS-executed selectively later, and TrustJS-native only after its native-validation gates exist**. Choosing this option does not remove the Rust daemon, aterm, crypto, or other successful native subsystems. It changes how new and changing business logic is authored and admitted.

Phase T0 – define a TrustJS-ready business island now

Keep production execution in Node/Electron and keep writing features in TypeScript. Move eligible logic into packages with explicit constraints:

- deterministic, provider-neutral functions or reducers;
- no DOM, Electron, filesystem, Git process, network, clock, random, locale, or ambient environment access;
- effects represented as typed inputs, outputs, or commands handled by a host adapter;
- bounded execution and explicit error values;
- no dependence on module-level singleton state.

Good candidates are policy calculations, normalizers, serializers, reducers, provider-neutral review/workspace decisions, and view-model projections. React components, Electron services, SSH transport, Git execution, PTY handling, and provider SDK adapters are not candidates.

Use the existing coarse `orca-dispatch` idea as the architectural precedent, but version the new ABI. Either constrain v1 values to a validated JSON-safe domain or use a tagged value encoding. Plain JSON silently loses JavaScript distinctions such as `undefined`, `NaN`, `-0`, BigInt, and lone UTF-16 surrogates.

Pin and hash the TS-to-JS transform. Prefer erasable TypeScript and a modern JS target; avoid TS constructs whose emitted helper/runtime semantics obscure the source contract.

Gate T0: selected modules are effect-free by enforced dependency rules, have a versioned ABI, run unchanged under normal Node/Electron, and carry source/emitted-JS identity plus cross-platform tests. No TrustJS claim is made yet.

Phase T1 – productize TrustJS as a consumable engine

This is work in Trust, not a path dependency from Orca into an entire mutable rustc checkout. Produce a small, versioned, pinned runtime artifact with:

- persistent engine lifetime rather than one 32 MiB thread per call;
- module/function invocation or an equivalent registered-export API;
- tagged input/output values and typed `NoCoverage`/fault responses;
- time, recursion, heap, output, and cancellation limits;
- deterministic engine identity and evidence metadata;
- configurable Node/Bun oracle discovery;
- clean locked builds and continuous macOS/Linux/Windows tests;
- cold-start, throughput, memory, and bundle-size benchmarks;
- a standalone/process-isolated runner first, followed by Node-API and WASM bindings only if their measurements justify them.

Land the pending S1d evidence before using those semantics. Continue the M1 ratchet: every admitted behavior is

exact or refused, never guessed.

Gate T1: a clean pinned artifact builds reproducibly on all Orca targets, has published coverage and performance evidence, and exposes a stable callable ABI. This still makes no proof or sandbox claim.

Phase T2 – shadow selected Orca exports

For each T0 module, run the emitted JavaScript through both the ordinary Node engine and the TrustJS runner in CI and development replay jobs. Node remains authoritative. Compare tagged results and effects over:

- existing unit and integration cases;
- generated/property and mutation cases;
- recorded, privacy-reviewed production shapes;
- boundary strings, Unicode, numeric corners, malformed data, and resource limits;
- macOS, Linux, Windows, WSL, and SSH-relevant data shapes.

`NoCoverage`, timeout, or engine fault means "not admitted; stay on Node." A behavioral divergence fails the admission gate and produces a minimized case; it is never silently classified by the source module itself.

Gate T2: zero divergence and zero `NoCoverage` for every admitted export on its complete pinned corpus, deterministic results on all target OSes, and negative controls that prove the comparator catches a wrong result.

Phase T3 – selective faithful-tier primary execution

Promote one pure module at a time only after T2. Use the same source and ABI so rollback is engine selection, not a data migration or hand-maintained code fork. Retain sampled Node shadow evaluation for a bounded rollout window.

Promotion must be context-aware:

- Electron main/CLI may use a process-isolated or Node-API artifact;
- renderer and web require WASM;
- an export needed on an SSH host stays on remote Node until a portable WASM artifact or explicitly supported remote binary exists;
- no module may assume the desktop architecture when its feature runs through the relay.

Gate T3: module ABI stability, acceptable measured latency/memory/startup, packaging and signing on every applicable platform, crash/resource isolation, two release-equivalent windows of shadow parity, and an exercised atomic rollback.

Phase T4 – effectful runtime adoption after TrustJS M2

Do not move async orchestration merely because pure functions work. Wait for TrustJS's actual M2 gate: reactor and Promise semantics, module graph, the admitted pure-JS `node:` surface, capability enforcement, async/module differential ledgers, and reproducible cross-platform packaging.

Even then, keep Electron DOM/rendering, native add-ons, PTYs, filesystem, Git, SSH, network, and provider APIs behind explicit host ports. Consider moving only orchestration whose effects can be injected and audited.

Gate T4: Trust's published M2 exit conditions plus Orca-specific disconnect, cancellation, retry, provider, WSL, SSH, and resource tests.

Phase T5 – autoformalized/native tier after TrustJS M4

At the future M4 boundary, TypeScript/JavaScript remains zero-authority input. TrustJS may produce an inspectable Rust+Lean/Trust-IR artifact and validated native installation candidate. Replace a manual Rust port only when the generated artifact clears both independent fidelity evidence and its stated proof/certificate gate.

The maintained source should remain TypeScript/JavaScript; generated Rust/Trust-IR should be reproducible build artifacts, not a second source tree edited by hand. That is the feature-velocity payoff of option 2.

6. Recommended ownership policy

Adopt option 2 as the strategic default with an explicit option-1 carve-out:

New or changing work	Default home
React, UI behavior, Electron integration, editor/browser views	TypeScript
Pure business rules, reducers, normalizers, provider-neutral planning	TypeScript/JavaScript in the TrustJS-ready island
Filesystem/network/provider/Git/SSH effects and SDK adaptation	TypeScript host ports unless a native boundary is independently justified
PTY, terminal engine/rendering, crypto, binary protocols, hot parsers	Rust
Stable, measured, privileged, memory-sensitive state machines	Rust or future admitted TrustJS-native artifact
SSH-host pure rules	TS now; TrustJS only with portable WASM/remote support

The operating rules are:

1. **Stop broad manual business-logic porting.** A new Rust port needs a measured performance, security, platform, or stability reason.
2. **Do not unwind successful Rust infrastructure.** The daemon, aterm, crypto, protocol, and established portable-core work are the substrate for either strategy.
3. **Keep shipping features in TypeScript.** Shape eligible code so it can be admitted by TrustJS later without blocking the feature now.
4. **Treat `NoCoverage` as a routing fact.** It means stay on Node, not waive the case or infer equivalence.
5. **Keep proof language precise.** Node/TrustJS equality is bounded differential evidence. TrustJS source and TS annotations have zero proof authority. Native/proof claims wait for the actual Trust-IR/Clean lane.
6. **Preserve all execution hosts.** Desktop, renderer, web, CLI, WSL, SSH, macOS, Linux, and Windows must be part of module admission.
7. **Keep upstream mergeability visible.** Prefer stable seams and retained high-level source over fork-wide semantic rewrites.

7. Immediate next decisions

1. Ratify the ownership table above as the default rule for new work.
2. Produce a generated as-built port inventory: `wired`, `shadow-only`, `test-only`, `prototype`, or `target` for every Rust crate/module.
3. Fix the generic packaging graph so every package build rebuilds or verifies the exact daemon artifact it ships.
4. Select three small T0 pilots: one normalizer, one reducer/state transition, and one provider-neutral policy calculation with strong existing tests.
5. Define `orca-business-abi-v1`, including tagged/safe values, errors, resource limits, artifact identity, and capability-free declarations.
6. In Trust, finish and publish the S1d gate, then design the persistent callable runner and cross-platform artifact. Do not link Orca directly to `~/trust`.
7. Add shadow comparison only after T0/T1 gates; keep Node authoritative until each export independently passes T2 and T3.

This sequence preserves current feature velocity, continues to exploit Rust where it already wins, and turns TrustJS maturity into incremental optionality rather than a blocker or a leap of faith.

Evidence index

Key ALab sources:

- build and entry points:
  `package.json`
  `electron.vite.config.ts`
  `run-electron-vite-dev.mjs`
  `electron-builder.config.cjs`
- Electron/preload/renderer:
  `src/main/index.ts`
  `createMainWindow.ts`
  `src/preload/index.ts`
  `src/renderer/src/main.tsx`
- daemon:
  `daemon-init.ts`
  `daemon-spawner.ts`
  `orca-daemon`
  `daemon protocol`
- terminal renderer:
  `aterm-strategy-select.ts`
  `aterm-shared-render-worker.ts`
  `aterm-render-worker.ts`
- Rust seams:
  `native/orca-node`
  `orca-dispatch`
  `orca-dispatch-seam.ts`
  `relay Git WASM`
- SSH:
  `relay.ts`
  `ssh-relay-deploy.ts`.

Reproducible upstream inspections at the pinned ref:

```
git show upstream/main:package.json
git show upstream/main:electron.vite.config.ts
git show upstream/main:src/main/daemon/daemon-init.ts
git show upstream/main:src/main/daemon/daemon-entry.ts
git show upstream/main:src/main/daemon/pty-subprocess.ts
git show upstream/main:src/main/daemon/headless-emulator.ts
git show upstream/main:src/renderer/src/lib/pane-manager/pane-dom-creation.ts
git show upstream/main:src/renderer/src/lib/pane-manager/pane-lifecycle.ts
```

Key Trust evidence at the pinned snapshot:

- `~/trust/docs/design/2026-07-20-trust-native-javascript-engine.md`;
- `~/trust/docs/design/2026-07-20-trustjs-m0-scope.md`;
- `~/trust/docs/design/2026-07-21-trustjs-m1-scope.md`;
- `~/trust/docs/design/2026-07-22-trustjs-m2-scope.md`;
- `~/trust/crates/trust-js-interp/src/lib.rs`;
- `~/trust/crates/trust-js-differential/src/heads.rs`;
- `~/trust/tests/js262/dashboard.md`;
- `~/trust/tests/js262/coverage.toml`.